Homework Solutions: LP & Dynamic Programming

Mohammad Abu Shariha

Problem 1. LP Formulation (spaceship oxidizers)

Problem statement (short). A spaceship has three compartments. Each compartment $i \in \{1, 2, 3\}$ uses identical small oxidizer units. Each unit for compartment i has independent probability of failure p_i (given). Units cannot be pre-tested; therefore backups are required. Per-unit characteristics (space, weight, and cost) and failure probabilities are:

Compartment i	Space (cu in)	Weight (lb)	Cost (\$)
p_i			
1	40	15	30,000
0.30			
2	50	20	35,000
0.40			
3	30	10	25,000
0.20			

Global limitations: total space ≤ 500 cu in, total weight ≤ 200 lb, total cost $\leq 400{,}000$\$.

Each compartment must have probability ≤ 0.05 that *all* its units fail. Goal: minimize the probability that all three compartments fail simultaneously.

Decision variables.

$$x_i = \text{number of oxidizer units placed in compartment } i, \quad i = 1, 2, 3.$$

We require $x_i \in \mathbb{Z}_{\geq 0}$ in reality. The formulation below treats them as continuous variables for the purpose of Linear Programming (relaxation), though the final implementation would require Integer Programming.

Modeling the objective. If units are independent and identically distributed per compartment, probability that all units in compartment i fail is

$$\Pr(\text{compartment } i \text{ all fail}) = p_i^{x_i}.$$

Probability that all three compartments fail simultaneously is

$$P_{\text{all}} = p_1^{x_1} \cdot p_2^{x_2} \cdot p_3^{x_3}.$$

Minimizing this product is equivalent to minimizing its natural logarithm. Because $\ln(\cdot)$ is monotone increasing, we minimize

$$\ln P_{\text{all}} = x_1 \ln p_1 + x_2 \ln p_2 + x_3 \ln p_3.$$

Note: Since $0 < p_i < 1$, the values $\ln p_i$ are negative. The objective function seeks to minimize a sum of negative terms (making them "more negative"), which correctly corresponds to driving the probabilities $p_i^{x_i}$ closer to zero.

Constraints.

- Space:

$$40x_1 + 50x_2 + 30x_3 \leq 500.$$

- Weight:
$$15x_1 + 20x_2 + 10x_3 \leq 200.$$
- Cost:
$$30{,}000x_1 + 35{,}000x_2 + 25{,}000x_3 \leq 400{,}000.$$
- Per-compartment reliability requirement: for each i,
$$p_i^{x_i} \leq 0.05 \quad \Longleftrightarrow \quad x_i \geq \frac{\ln 0.05}{\ln p_i}.$$

  Evaluate numerically:
$$x_1 \geq \frac{\ln 0.05}{\ln 0.3} \approx 2.488 \Rightarrow x_1 \geq 3,$$
$$x_2 \geq \frac{\ln 0.05}{\ln 0.4} \approx 3.269 \Rightarrow x_2 \geq 4,$$
$$x_3 \geq \frac{\ln 0.05}{\ln 0.2} \approx 1.861 \Rightarrow x_3 \geq 2.$$

Feasibility Check. We verify that the minimal integer solution derived from reliability bounds, $(x_1, x_2, x_3) = (3, 4, 2)$, satisfies the global resource limits:

- Space: $40(3) + 50(4) + 30(2) = 120 + 200 + 60 = 380 \leq 500$. **(OK)**
- Weight: $15(3) + 20(4) + 10(2) = 45 + 80 + 20 = 145 \leq 200$. **(OK)**
- Cost: $30k(3) + 35k(4) + 25k(2) = 90k + 140k + 50k = 280k \leq 400k$. **(OK)**

Thus, a feasible solution exists.

Complete (mixed-integer) LP formulation.

$$\begin{aligned}
\text{Minimize} \quad & z := (\ln p_1)x_1 + (\ln p_2)x_2 + (\ln p_3)x_3 \\
\text{subject to} \quad & 40x_1 + 50x_2 + 30x_3 \leq 500, \\
& 15x_1 + 20x_2 + 10x_3 \leq 200, \\
& 30000x_1 + 35000x_2 + 25000x_3 \leq 400000, \\
& x_1 \geq \frac{\ln 0.05}{\ln 0.3}, \quad x_2 \geq \frac{\ln 0.05}{\ln 0.4}, \quad x_3 \geq \frac{\ln 0.05}{\ln 0.2}, \\
& x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0}.
\end{aligned}$$

Notes. Numeric constants: $\ln p_1 \approx -1.20397,\ \ln p_2 \approx -0.91629,\ \ln p_3 \approx -1.60944.$

Problem 2 (a). LP Conversion to Standard Form 3

Original problem (a).

$$\begin{aligned} \text{Maximize} \quad & z = 3x_1 + 5x_2 - 2x_3 \\ \text{subject to} \quad & 2x_1 - x_2 + x_3 \leq 10, \\ & -x_1 + 3x_2 + 2x_3 \geq 8, \\ & x_1 - 2x_2 + x_3 \geq -6, \\ & x_2, x_3 \geq 0. \end{aligned}$$

Goal: convert to Standard Form 3 - all constraints as equalities, all variables nonnegative, objective as a maximization, and RHS constants non-negative.

Step 1: Handle RHS signs. The third constraint has a negative RHS: $x_1 - 2x_2 + x_3 \geq -6$. Multiply by -1:

$$-x_1 + 2x_2 - x_3 \leq 6.$$

Step 2: Slack/surplus variables.

$$\begin{aligned} 2x_1 - x_2 + x_3 + s_1 &= 10, \qquad s_1 \geq 0 \quad (\text{slack for } \leq \text{ row}), \\ -x_1 + 3x_2 + 2x_3 - s_2 &= 8, \qquad s_2 \geq 0 \quad (\text{surplus for } \geq \text{ row}), \\ -x_1 + 2x_2 - x_3 + s_3 &= 6, \qquad s_3 \geq 0 \quad (\text{slack for the converted } \leq \text{ row}). \end{aligned}$$

Step 3: Free variable handling. The variable x_1 is not constrained to be nonnegative in the statement (no sign given). Represent x_1 as difference of two nonnegatives:

$$x_1 = x_1^+ - x_1^-, \qquad x_1^+ \geq 0,\ x_1^- \geq 0.$$

We assume $x_2, x_3 \geq 0$ as given.

Step 4: Substitute x_1 and write equalities.

$$\begin{aligned} 2(x_1^+ - x_1^-) - x_2 + x_3 + s_1 &= 10, \\ -(x_1^+ - x_1^-) + 3x_2 + 2x_3 - s_2 &= 8, \\ -(x_1^+ - x_1^-) + 2x_2 - x_3 + s_3 &= 6. \end{aligned}$$

All variables are now nonnegative:

$$x_1^+, x_1^-, x_2, x_3, s_1, s_2, s_3 \geq 0.$$

Objective in nonnegative variables:

$$\max\ z = 3(x_1^+ - x_1^-) + 5x_2 - 2x_3.$$

Remark on Initial BFS: The equality derived from the second constraint ($-x_1 + 3x_2 + 2x_3 - s_2 = 8$) contains a negative surplus variable and therefore lacks an immediate basic variable. To solve this system via Simplex, one must add an artificial variable to this row and employ the Two-Phase Method or Big-M Method to obtain an initial Basic Feasible Solution.

Problem 2 (b). LP Conversion to Standard Form 3

Original problem (b).

$$\begin{aligned} \text{Maximize} \quad & z = 4x_1 + x_2 + 6x_3 \\ \text{subject to} \quad & x_1 + 2x_2 \leq 14, \\ & 3x_1 + x_2 - x_3 \geq 5, \\ & x_2 - x_3 \leq -2, \\ & x_1, x_3 \geq 0. \end{aligned}$$

Notes: Variable x_2 is not specified nonnegative in the statement so treat x_2 as free. We are given $x_1, x_3 \geq 0$.

Step 1: handle RHS signs. The third constraint has a negative RHS: $x_2 - x_3 \leq -2$. Multiply by -1:

$$-x_2 + x_3 \geq 2.$$

Step 2: Slack/surplus variables.

$$\begin{aligned} x_1 + 2x_2 + s_1 &= 14, & s_1 &\geq 0, \\ 3x_1 + x_2 - x_3 - s_2 &= 5, & s_2 &\geq 0, \\ -x_2 + x_3 - s_3 &= 2, & s_3 &\geq 0. \end{aligned}$$

Step 3: Represent free variable x_2 as

$$x_2 = x_2^+ - x_2^-, \qquad x_2^+, x_2^- \geq 0.$$

Step 4: Substitute and write equalities.

$$\begin{aligned} x_1 + 2(x_2^+ - x_2^-) + s_1 &= 14, \\ 3x_1 + (x_2^+ - x_2^-) - x_3 - s_2 &= 5, \\ -(x_2^+ - x_2^-) + x_3 - s_3 &= 2. \end{aligned}$$

All variables nonnegative:

$$x_1, x_3, x_2^+, x_2^-, s_1, s_2, s_3 \geq 0.$$

Objective in nonnegative variables:

$$\max \; z = 4x_1 + (x_2^+ - x_2^-) + 6x_3.$$

Remark on initial BFS: The equalities derived from the $\geq$ constraints (rows 2 and 3) contain negative surplus variables and lack immediate basic variables. To proceed with Simplex, one must add **artificial variables** to these rows and use the **Two-Phase Method** or **Big-M Method** to find an initial Basic Feasible Solution.

Problem 3. Simplex Method (solve the LP)

problem statement.

$$\begin{aligned} \text{Maximize} \quad & z = 10x_1 + 8x_2 + 5x_3 \\ \text{subject to} \quad & x_1 + x_2 + 2x_3 \leq 20, \\ & 3x_1 + x_2 + x_3 \leq 30, \\ & x_1 + 2x_2 - 3x_3 \leq 18, \\ & x_1, x_2, x_3 \geq 0. \end{aligned}$$

Step 1. Initial Tableau. Add slack variables $s_1, s_2, s_3 \geq 0$.

Basis	x_1	x_2	x_3	s_1	s_2	s_3	RHS
s_1	1	1	2	1	0	0	20
s_2	**3**	1	1	0	1	0	30
s_3	1	2	-3	0	0	1	18
z	-10	-8	-5	0	0	0	0

Step 2. Pivot 1. Most negative coefficient in z-row is -10 (x_1 enters). Ratios: $s_1 : 20/1 = 20$, $s_2 : 30/3 = 10$ (winner), $s_3 : 18/1 = 18$. Pivot on s_2 row, column x_1.

- New $R_2 \leftarrow R_2/3$
- $R_1 \leftarrow R_1 - 1(NewR_2)$
- $R_3 \leftarrow R_3 - 1(NewR_2)$
- $R_z \leftarrow R_z + 10(NewR_2)$

Basis	x_1	x_2	x_3	s_1	s_2	s_3	RHS
s_1	0	2/3	5/3	1	$-1/3$	0	10
x_1	1	1/3	1/3	0	1/3	0	10
s_3	0	**5/3**	$-10/3$	0	$-1/3$	1	8
z	0	$-14/3$	$-5/3$	0	10/3	0	100

Step 3. pivot 2. most negative coefficient in z-row is $-14/3$ (x_2 enters). Ratios: $s_1 : 10/(2/3) = 15$, $x_1 : 10/(1/3) = 30$, $s_3 : 8/(5/3) = 24/5 = 4.8$ (winner). Pivot on s_3 row, column x_2.

- New $R_3 \leftarrow R_3 \times (3/5)$
- $R_1 \leftarrow R_1 - (2/3)(NewR_3)$
- $R_2 \leftarrow R_2 - (1/3)(NewR_3)$
- $R_z \leftarrow R_z + (14/3)(NewR_3)$

Basis	x_1	x_2	x_3	s_1	s_2	s_3	RHS
s_1	0	0	**3**	1	$-1/5$	$-2/5$	34/5
x_1	1	0	1	0	2/5	$-1/5$	42/5
x_2	0	1	-2	0	$-1/5$	3/5	24/5
z	0	0	-11	0	12/5	14/5	612/5

Step 4. Pivot 3. Most negative coefficient in z-row is -11 (x_3 enters). Ratios: $s_1 : (34/5)/3 = 34/15 \approx 2.26$ (winner), $x_1 : (42/5)/1 = 8.4$. x_2 has negative coeff, ignore. Pivot on s_1 row, column x_3.

- New $R_1 \leftarrow R_1/3$
- $R_2 \leftarrow R_2 - 1(NewR_1)$
- $R_3 \leftarrow R_3 + 2(NewR_1)$
- $R_z \leftarrow R_z + 11(NewR_1)$

Basis	x_1	x_2	x_3	s_1	s_2	s_3	RHS
x_3	0	0	1	1/3	−1/15	−2/15	34/15
x_1	1	0	0	−1/3	7/15	−1/15	92/15
x_2	0	1	0	2/3	−1/3	1/3	28/3
z	0	0	0	11/3	5/3	4/3	442/3

conclusion and optimality Check. All coefficients in the objective row (the reduced costs) are non-negative ($11/3, 5/3, 4/3 \geq 0$). Therefore, the current basic feasible solution is optimal.

$$\boxed{x_1 = \frac{92}{15}, \quad x_2 = \frac{28}{3}, \quad x_3 = \frac{34}{15}, \quad z = \frac{442}{3} \approx 147.33.}$$

Problem 4. 3-Dense Subsequence (dynamic programming)

Problem restatement. Given sequence $S = (a_1, a_2, \ldots, a_n)$. A subsequence $A \subseteq \{1, \ldots, n\}$ is called *3-dense* if for every triple of consecutive original indices $(i, i+1, i+2)$ at least one index among them is selected (i.e., we are not allowed to skip three consecutive indices). Goal: compute a 3-dense subsequence with maximum possible sum.

Observation. Equivalent: in the chosen subsequence we cannot have three consecutive *unselected* elements.

DP state. Define

$dp[i][k]$ = maximum sum obtainable considering prefix $a_1, \ldots, a_i$ where the last k elements were skipped,

with $k \in \{0, 1, 2\}$. Here:

- $k = 0$: the i-th element *is selected*.
- $k = 1$: the i-th element is skipped and the final run of consecutive skips has length 1.
- $k = 2$: the final run of consecutive skips has length 2.

We *forbid* $k = 3$.

Initialization.

$$dp[0][0] = 0, \qquad dp[0][1] = dp[0][2] = -\infty.$$

Transitions (for $i \geq 0$ to $i+1$).

- If we *select* a_{i+1}:
$$dp[i+1][0] = \max_{k \in \{0,1,2\}} \big(dp[i][k] + a_{i+1}\big).$$
- If we *skip* a_{i+1}:
$$dp[i+1][1] = \max(dp[i+1][1],\ dp[i][0]) \quad \text{(last run becomes length 1)},$$
$$dp[i+1][2] = \max(dp[i+1][2],\ dp[i][1]) \quad \text{(last run becomes length 2)}.$$
(We cannot skip when the previous run length was 2.)

Correctness and Edge Cases.

- **Induction:** Base case ($i = 0$) is trivially correct. Assume states at i correctly store max sums for run-lengths of 0, 1, or 2 skips. The transitions for $i+1$ exhaustively cover the two options (select or skip). If we skip, we strictly increment the run length ($0 \to 1, 1 \to 2$). If the run length is already 2, the "skip" transition is invalid, enforcing the 3-dense constraint.
- **Edge Cases ($n < 3$):** If $n = 1$ or $n = 2$, it is impossible to skip 3 consecutive elements. The DP naturally handles this: it will compute the optimal sum allowing skips up to length 2, effectively permitting any subset selection.

- **Negative Values:** The logic holds for negative a_i because we use maximization. The sentinel $-\infty$ ensures invalid states (like skipping 3 times) are never chosen.

Answer.

$$\text{Maximum achievable sum} = \max\{dp[n][0], dp[n][1], dp[n][2]\}.$$

O(n) implementation (space O(1) rolling).

```
function max3dense(S[1..n]):
    NEG = -infty
    dp0 = 0       # last chosen
    dp1 = NEG     # last one skipped
    dp2 = NEG     # last two skipped

    for i in 1..n:
        new0 = max(dp0, dp1, dp2) + S[i]   # choose a_i
        new1 = dp0                         # skip a_i, now run length 1
        new2 = dp1                         # skip a_i, now run length 2
        dp0, dp1, dp2 = new0, new1, new2

    return max(dp0, dp1, dp2)
```

Reconstruction. To recover indices chosen, store predecessor pointers for each i and k (which previous state achieved the maximum) and backtrack from the final state achieving the maximum.
Complexity. $O(n)$ time, $O(1)$ extra space (or $O(n)$ if you store parents to reconstruct the subsequence).

Problem 5. Minimum-Cost Triangulation of a Convex Polygon (dynamic programming)

Problem restatement. Given a convex polygon P with vertices $v_1, \ldots, v_n$ listed counterclockwise. For any triple $i < j < k$ we are given a nonnegative cost $c(i, j, k)$ of forming triangle (v_i, v_j, v_k). A triangulation partitions P into $n - 2$ triangles using noncrossing diagonals. Find a triangulation minimizing total cost.

Classical DP formulation. Let

$$dp[i][j] = \text{minimum cost to triangulate the subpolygon with vertices } v_i, v_{i+1}, \ldots, v_j,$$

for $1 \le i < j \le n$. For subpolygons with fewer than 3 vertices (i.e., $j - i < 2$), $dp[i][j] = 0$ (no triangle).

Recurrence. For $j - i \ge 2$,

$$dp[i][j] = \min_{k=i+1,\ldots,j-1} \Big\{ dp[i][k] + dp[k][j] + c(i, k, j) \Big\}.$$

Choose k as the vertex that forms the triangle (v_i, v_k, v_j) splitting the polygon into two subproblems.

Base cases.

$$dp[i][i] = 0, \qquad dp[i][i + 1] = 0.$$

Top-down /bottom-up algorithm (bottom up shown).

```
function minTriangulationCost(n, c):
    for i in 1..n:
        dp[i][i] = 0
        dp[i][i+1] = 0

    for length = 2..(n-1):        # length = j - i
        for i = 1..n-length:
            j = i + length
            dp[i][j] = +infty
            for k = i+1..j-1:
                cost = dp[i][k] + dp[k][j] + c(i,k,j)
                if cost < dp[i][j]:
                    dp[i][j] = cost
                    parent[i][j] = k

    return dp[1][n]
```

Complexity. $O(n^3)$ time, $O(n^2)$ space. Reconstruct triangulation using the recorded parent indices.

remark. we assume the cost function $c(i, k, j)$ (eg., area or perimeter of triangle (v_i, v_k, v_j)) can be computed in $O(1)$ time given the vertex coordinates. This ensures the total time remains $O(n^3)$.